

Rapport de projet fin d'année

3^e année Ingénierie Informatique et Réseaux

Sous le thème

APPLICATION WEB DE GESTION DE LOCATION DE VOITURES

Réalisé par :

Kawtar Benyoussef

Encadré par :

Pr. Houda Orchi

ANNEE UNIVERSITAIRE : 2025–2026

Dédicaces

Nous dédions ce travail à nos familles, pour leur soutien constant, leur patience et leurs encouragements tout au long de notre parcours académique.

Nous le dédions également à nos enseignants, dont les conseils et l'accompagnement ont contribué à renforcer nos compétences techniques, méthodologiques et professionnelles.

Enfin, nous dédions ce projet à toutes les personnes qui nous ont aidés, directement ou indirectement, dans la réalisation de cette application web de gestion de location de voitures.

Remerciements

Avant de présenter notre projet de fin d'année, nous souhaitons exprimer notre gratitude envers notre encadrant, Pr. Houda Orchi, pour ses orientations, ses remarques et son suivi durant les différentes étapes du projet.

Nous remercions également l'ensemble du corps professoral et administratif pour la qualité de la formation reçue, ainsi que pour l'environnement pédagogique mis à notre disposition.

Nos remerciements s'adressent aussi à nos collègues pour leurs échanges, leurs retours et leur collaboration. Ces contributions nous ont permis d'améliorer progressivement la conception, la réalisation et la validation de notre solution.

Résumé

Le présent rapport décrit la conception et la réalisation d'une application web de gestion de location de voitures destinée à une agence de taille moyenne. Le projet répond au besoin de centraliser la gestion du parc automobile, des clients, des demandes de réservation, du suivi des locations et des indicateurs d'activité.

La solution développée repose sur le framework Django, une base de données SQLite et des interfaces construites avec les templates Django et Bootstrap. Elle intègre plusieurs modules : authentification avec récupération de mot de passe, gestion des voitures avec images, gestion des clients, cycle de vie des réservations, catalogue et espace client, gestion des rôles, statistiques, exports CSV, génération de factures PDF et rapport global.

L'application distingue trois profils principaux : administrateur, employé et client. L'administrateur dispose d'un accès complet, l'employé traite les opérations courantes, tandis que le client peut créer un compte, consulter le catalogue, déposer une demande de réservation et suivre son historique. Des règles métier assurent la cohérence du système, notamment le contrôle des conflits de réservation, l'interdiction de louer une voiture en maintenance, l'annulation limitée aux demandes autorisées et le calcul automatique du montant total.

Ce rapport présente le contexte du projet, l'étude de l'existant, l'analyse fonctionnelle, la modélisation UML, l'architecture technique, les choix de réalisation, les tests effectués et les perspectives d'amélioration.

Mots clés : Django, location de voitures, réservation, gestion de parc, application web, SQLite, Bootstrap.

Abstract

This report presents the design and implementation of a car rental management web application for a medium-sized agency. The project aims to centralize vehicle management, customer records, booking requests, rental tracking and business indicators.

The application is built with the Django framework, an SQLite database and user interfaces based on Django templates and Bootstrap. It includes several modules : authentication with password reset, vehicle management with images, customer management, reservation lifecycle, customer catalog and dashboard, role management, statistics, CSV exports, PDF invoice generation and a global PDF report.

The system defines three main profiles : administrator, employee and customer. The administrator has full access, the employee handles daily operations, while the customer can create an account, browse the catalog, submit a reservation request and consult reservation history. Business rules ensure data consistency, including reservation conflict detection, prevention of renting vehicles under maintenance, controlled cancellation and automatic computation of the total amount.

This report details the project context, existing solutions, functional analysis, UML modeling, technical architecture, implementation choices, validation tests and possible improvements.

Keywords : Django, car rental, reservation, fleet management, web application, SQLite, Bootstrap.

Liste des abréviations

Abréviation	Signification
AJAX	Asynchronous JavaScript and XML, utilisé ici pour les requêtes asynchrones renvoyant du JSON.
CSV	Comma-Separated Values, format d'export tabulaire.
DH	Dirham marocain, devise utilisée dans l'application.
MVT	Model-View-Template, architecture logique de Django.
PDF	Portable Document Format, utilisé pour les factures et le rapport global.
PFA	Projet de fin d'année.
RBAC	Role-Based Access Control, gestion des permissions par rôle.
SQLite	Système de gestion de base de données embarqué utilisé par le projet.
UML	Unified Modeling Language, langage de modélisation utilisé pour les schémas.

Table des matières

Dédicaces	i
Remerciements	ii
Résumé	iii
Abstract	iv
Liste des abréviations	v
Introduction générale	1
1 Contexte et problématique	2
1.1 Contexte du projet	2
1.2 Problématique	2
1.3 Étude synthétique de l'existant	3
1.4 Périmètre fonctionnel	3
1.5 Objectifs du projet	4
1.6 Résultats obtenus	4
1.7 Gestion de projet	5
1.7.1 Phases du projet	5
1.7.2 Planning synthétique	5
1.7.3 Outils utilisés	5
1.7.4 Organisation du travail	6
2 Analyse des besoins	7
2.1 Acteurs du système	7
2.2 Besoins fonctionnels	7
2.3 Matrice des droits	8

2.4	Règles métier	9
2.5	Scénarios métiers principaux	9
2.5.1	Demande de réservation côté client	9
2.5.2	Traitement d'une demande côté personnel	9
2.5.3	Pilotage administratif	10
2.6	Fonctionnalités non retenues	10
2.7	Sécurité et contrôle des accès	10
3	Conception et architecture	11
3.1	Architecture logicielle retenue	11
3.2	Structure logicielle du projet	12
3.3	Modèle de données	12
3.4	Diagramme de classes	13
3.5	Modèle logique de données	14
3.6	Schémas UML retenus	14
3.6.1	Diagramme de cas d'utilisation	15
3.6.2	Diagramme de séquence : demande de réservation	16
3.6.3	Diagramme de séquence : traitement par le personnel	17
3.6.4	Diagramme d'activité	18
3.6.5	Diagrammes techniques	19
3.7	Remarques de conception	21
4	Réalisation et fonctionnalités	22
4.1	Synthèse des fonctionnalités	22
4.2	Authentification	23
4.3	Gestion des voitures et des catégories	23
4.4	Gestion des clients	24
4.5	Réservations	25
4.6	Espace client	27
4.7	Gestion des rôles	30
4.8	Exports PDF et CSV	31
4.9	Statistiques	33
4.10	Paievements	34
4.11	Historique et notifications	34
4.12	Bilan de réalisation	35

5	Tests et validation	36
5.1	Stratégie de test	36
5.2	Résultat de la suite Django	36
5.3	Couverture fonctionnelle	37
5.4	Exemples de tests représentatifs	37
5.5	Tests end-to-end Playwright	37
5.6	Recette fonctionnelle	38
5.7	Avertissements et limites	38
5.8	Commandes de validation	38
6	Conclusion et perspectives	39
	Bibliographie	41
A	Annexes	43
A.1	Commandes d'installation et d'exécution	43
A.2	Identifiants de démonstration	43
A.3	Extrait des routes principales	44
A.4	Extrait de calcul métier	44
A.5	Exemple de contrôle de conflit	45
A.6	Extrait de journalisation	45
A.7	Structure du fichier requirements.txt	45
A.8	Variables d'environnement	46

Table des figures

3.1	Diagramme de classes complet du système	13
3.2	Diagramme de cas d'utilisation	15
3.3	Création d'une demande de réservation par le client	16
3.4	Traitement d'une réservation par le personnel	17
3.5	Cycle de vie d'une réservation	18
3.6	Diagramme de composants	19
3.7	Diagramme de déploiement	20
3.8	Architecture globale du système	21
4.1	Interface de connexion	23
4.2	Liste des voitures avec recherche et filtres	24
4.3	Liste des clients	25
4.4	Liste des réservations et actions de traitement	26
4.5	Détail d'une réservation avec actions, paiements et historique	27
4.6	Tableau de bord client	28
4.7	Catalogue des voitures côté client	29
4.8	Vérification de disponibilité côté client	30
4.9	Gestion des rôles du personnel	31
4.10	Facture PDF générée pour une réservation	32
4.11	Rapport PDF global	33
4.12	Page des statistiques	34

Liste des tableaux

1.1	Comparaison synthétique des approches	3
1.2	Correspondance entre objectifs et résultats obtenus	4
1.3	Planning synthétique du projet	5
1.4	Outils et technologies du projet	6
2.1	Acteurs du système	7
2.2	Besoins fonctionnels par module	7
2.3	Matrice synthétique des droits fonctionnels	8
2.4	Règles métier principales	9
3.1	Choix technologiques essentiels	11
3.2	Entités métier et responsabilités	12
3.3	Modèle logique de données	14
4.1	Fonctionnalités réalisées par module	22
5.1	Couverture synthétique des tests Django	37
5.2	Exemples de tests représentatifs	37
5.3	Couverture synthétique des tests Playwright	38
A.1	Comptes de démonstration	44

Introduction générale

La transformation numérique des activités de service touche directement les agences de location de voitures. Ces structures ont besoin d'un suivi fiable des véhicules, des clients, des demandes de réservation, des locations actives et des documents produits à partir de ces opérations. En l'absence d'un système centralisé, la gestion devient vite répétitive, difficile à contrôler et source d'erreurs.

Le projet présenté dans ce rapport consiste à concevoir et réaliser une application web de gestion de location de voitures développée avec Django. L'objectif est à la fois de proposer une solution fonctionnelle et de produire un rapport académique rigoureux, cohérent avec l'implémentation réalisée et adapté à une soutenance de projet de fin d'année.

Le rapport est organisé comme suit :

- le premier chapitre présente le contexte, la problématique et le périmètre du projet, ainsi que l'organisation de la démarche ;
- le deuxième chapitre formalise les besoins fonctionnels, les acteurs, les règles métier et les mécanismes de sécurité ;
- le troisième chapitre traite de la conception et de l'architecture, avec les schémas UML et le modèle de données ;
- le quatrième chapitre détaille la réalisation et l'ensemble des fonctionnalités implémentées ;
- le cinquième chapitre présente les tests, la validation et les limites identifiées ;
- enfin, la conclusion synthétise les apports du projet et les perspectives d'évolution.

Chapitre 1

Contexte et problématique

Ce chapitre présente le contexte dans lequel s’inscrit le projet, la problématique traitée ainsi que le périmètre fonctionnel retenu. Il introduit également l’organisation de la démarche projet, les outils utilisés et le planning de réalisation.

1.1 Contexte du projet

Le projet porte sur une application web de gestion de location de voitures destinée à une agence de taille moyenne. L’objectif général est de centraliser des opérations qui, sans outil dédié, sont souvent réparties entre des fichiers manuels, des tableurs, des échanges téléphoniques et des documents peu homogènes.

Le besoin initial couvrait l’authentification, la gestion des voitures, des clients, des réservations et un tableau de bord. L’évolution du projet a ensuite enrichi ce socle avec un espace client, des exports CSV, des documents PDF, des statistiques et une gestion des rôles du personnel.

1.2 Problématique

Une agence de location doit répondre à plusieurs questions critiques au quotidien :

- quelles voitures sont disponibles à une période donnée ;
- quel client est lié à une réservation ;
- quel montant doit être calculé pour une location ;
- quel est l’état d’avancement d’une demande ou d’une location ;
- quels indicateurs résument l’activité de l’agence.

Sans système structuré, ces questions génèrent plusieurs risques : double réservation d’un même véhicule, erreur sur les dates, suivi incomplet de l’historique, difficulté à distinguer les droits du personnel et faible traçabilité documentaire.

La problématique centrale du projet peut donc être formulée ainsi : comment concevoir une application web simple, maintenable et fiable permettant de gérer le parc automobile, les clients, les réservations et les documents associés, tout en séparant correctement les rôles utilisateur ?

1.3 Étude synthétique de l'existant

TABLE 1.1 – Comparaison synthétique des approches

Approche	Avantages	Limites
Gestion manuelle	Mise en place immédiate, aucun coût technique.	Erreurs fréquentes, faible traçabilité, absence de contrôle automatisé.
Tableurs	Calculs simples et classement initial des données.	Peu adaptés à la sécurité, au multi-rôle et aux conflits de réservation.
Logiciels spécialisés	Fonctionnalités riches et cadrées.	Coût, complexité, dépendance à une solution externe.
Application développée	Adaptée au besoin réel et évolutive.	Demande analyse, développement, validation et maintenance.

1.4 Périmètre fonctionnel

L'application développée couvre les modules suivants :

- authentification et redirection selon le profil ;
- récupération de mot de passe ;
- gestion des voitures avec image optionnelle ;
- gestion des clients ;
- cycle de vie complet des réservations ;
- espace client avec catalogue et demandes de réservation ;
- actions asynchrones AJAX ;
- gestion des rôles du personnel ;
- statistiques, exports CSV, factures PDF et rapport PDF global ;
- pages d'erreur, commande de seed et suites de tests.

Les fonctionnalités suivantes n'ont pas été implémentées dans cette version :

- paiement en ligne (seul le paiement enregistré manuellement est implémenté) ;
- notification externe par email ou SMS ;

- gestion multi-agences ;
- déploiement cloud.

1.5 Objectifs du projet

Les objectifs du projet sont les suivants :

- centraliser les données métier dans une application web unique ;
- automatiser le calcul du montant total et le contrôle des conflits de réservation ;
- distinguer les accès administrateur, employé et client ;
- améliorer l'exploitation des données via exports, PDF et statistiques ;
- proposer une base maintenable, testée et documentable dans un cadre académique.

1.6 Résultats obtenus

Le tableau suivant synthétise les résultats concrets apportés par l'application par rapport aux objectifs fixés :

TABLE 1.2 – Correspondance entre objectifs et résultats obtenus

Objectif initial	Fonctionnalité réalisée	Résultat obtenu
Centraliser les données	Application web unique avec 7 modèles de données	Les informations véhicules, clients, réservations, paiements et catégories sont regroupées dans une seule base.
Automatiser les calculs	Calcul automatique du montant total et contrôle des conflits	Le montant est recalculé à chaque sauvegarde ; les conflits de réservation sont bloqués automatiquement.
Séparer les rôles	Trois profils (Admin, Employé, Client) avec groupes Django	Chaque utilisateur n'accède qu'aux fonctionnalités qui lui sont dédiées.
Exploiter les données	Exports CSV, factures PDF, rapport PDF, statistiques	Les données saisies sont transformées en documents exploitables et indicateurs de pilotage.
Garantir la fiabilité	222 tests Django et 35 tests Playwright	Les comportements critiques sont vérifiés automatiquement à chaque évolution.
Assurer la traçabilité	Historique des actions et notifications persistantes	Chaque action importante est journalisée et l'utilisateur est informé.

1.7 Gestion de projet

1.7.1 Phases du projet

Le projet a été déroulé selon une démarche structurée en cinq phases principales :

1. **Analyse** : étude de l'existant, recueil des besoins fonctionnels et non fonctionnels, identification des acteurs et des règles métier.
2. **Conception** : modélisation UML (diagrammes de cas d'utilisation, de classes, de séquence, d'activité et de composants), définition de l'architecture technique et du modèle de données.
3. **Développement** : implémentation des modèles, formulaires, vues, routes, templates, exports CSV, génération PDF et actions AJAX, selon l'architecture MVT de Django.
4. **Tests** : rédaction et exécution de 222 tests Django (unitaires et fonctionnels) ainsi que de 35 tests end-to-end Playwright.
5. **Finalisation** : rédaction du rapport, compilation LaTeX via Overleaf, production des diagrammes UML et captures d'écran.

1.7.2 Planning synthétique

Le tableau suivant synthétise la répartition des activités sur la durée du projet :

TABLE 1.3 – Planning synthétique du projet

Phase	Activités principales	Durée indicative
Analyse	Étude de l'existant, recueil des besoins, règles métier	2 semaines
Conception	Diagrammes UML, modèle de données, architecture technique	2 semaines
Développement	Implémentation Django, CRUD, espace client, AJAX, PDF, CSV	6 semaines
Tests	Tests Django (222), tests Playwright (35), recette fonctionnelle	2 semaines
Finalisation	Rapport LaTeX, captures d'écran, compilation, relecture	2 semaines

1.7.3 Outils utilisés

Les outils et technologies mobilisés tout au long du projet sont les suivants :

TABLE 1.4 – Outils et technologies du projet

Outil	Rôle dans le projet
Python [13]	Langage principal de l'application.
Django 6.0.5 [1]	Framework web : modèles, vues, formulaires, authentification, routes.
SQLite [5]	Base de données relationnelle embarquée, utilisée pour le développement.
Bootstrap 5 [6]	Framework CSS pour la mise en forme responsive des interfaces.
ReportLab [7]	Bibliothèque Python pour la génération de factures PDF et du rapport global.
Pillow	Gestion du champ image des véhicules (<code>ImageField</code>).
Playwright [8]	Framework de tests end-to-end pour la validation des parcours utilisateur.
Git / GitHub	Gestion de version du code source et du dépôt projet.
Overleaf / LaTeX	Rédaction et compilation du rapport de projet.

1.7.4 Organisation du travail

Le développement a été réalisé de manière itérative : chaque nouvelle fonctionnalité a été implémentée, testée puis validée avant de passer à la suivante. Les commits Git ont permis de suivre l'évolution du code et de revenir à des états antérieurs en cas de besoin. La rédaction du rapport a été menée en parallèle du développement, chaque chapitre étant progressivement enrichi à mesure que les fonctionnalités étaient finalisées.

Les tests ont été écrits au fur et à mesure de l'implémentation, ce qui a permis de détecter rapidement les régressions et de maintenir une couverture fonctionnelle élevée. Les diagrammes UML ont été modélisés à partir du code réalisé, afin de garantir leur cohérence avec l'implémentation.

Le rapport a été rédigé en utilisant LaTeX via Overleaf, ce qui a facilité la gestion des références bibliographiques, des figures et de la mise en forme académique.

Chapitre 2

Analyse des besoins

Ce chapitre formalise les besoins fonctionnels, les acteurs, les droits d'accès et les règles métier retenues pour l'application. L'objectif est de préciser ce que le système doit permettre sans détailler à ce stade les choix d'implémentation.

2.1 Acteurs du système

TABLE 2.1 – Acteurs du système

Acteur	Rôle principal
Administrateur	Gère les données de référence, les comptes du personnel, les paiements, les statistiques et les documents.
Employé	Consulte les données opérationnelles et traite les réservations au quotidien.
Client	Consulte le catalogue, crée des demandes de réservation, suit ses réservations et récupère ses factures.

2.2 Besoins fonctionnels

TABLE 2.2 – Besoins fonctionnels par module

Module	Besoins couverts
Authentification	Connexion, déconnexion, redirection selon le profil, protection des pages internes et récupération de mot de passe.
Voitures et catégories	Gestion du parc automobile, images optionnelles, statuts, catégories, recherche, filtres et export CSV.
Clients	Création, consultation, modification, suppression, recherche, historique des réservations et export CSV.

Réservations	Création, contrôle des dates et des conflits, acceptation, refus, annulation, finalisation, facture PDF et export CSV.
Espace client	Inscription, catalogue, détail d'une voiture, demande de réservation, vérification AJAX et suivi personnel.
Administration	Tableau de bord, statistiques, rapport PDF, gestion des rôles et données de démonstration.
Paielements	Enregistrement manuel, modification, consultation et filtrage par statut.
Traçabilité	Historique des actions importantes, notifications internes et modification du profil utilisateur.

2.3 Matrice des droits

TABLE 2.3 – Matrice synthétique des droits fonctionnels

Action ou module	Admin	Employé	Client
Tableau de bord et statistiques	Oui	Oui	Non
Gestion complète des voitures et catégories	Oui	Non	Non
Consultation du parc automobile	Oui	Oui	Oui, via catalogue
Gestion complète des clients	Oui	Partielle	Non
Création et traitement des réservations	Oui	Oui	Demande seulement
Paielements	Oui	Consultation	Non
Exports CSV, factures et rapport PDF	Oui	Oui	Factures propres
Historique et notifications	Oui	Oui	Notifications propres
Gestion des rôles du personnel	Oui	Non	Non
Profil personnel	Oui	Oui	Oui

2.4 Règles métier

TABLE 2.4 – Règles métier principales

Code	Règle
RG1	Toute page interne nécessite un utilisateur authentifié.
RG2	L'immatriculation d'une voiture, le CIN d'un client et le nom d'une catégorie sont uniques.
RG3	Une réservation est toujours liée à un client et à une voiture.
RG4	La date de fin doit être postérieure à la date de début lors d'une création contrôlée.
RG5	Une voiture en maintenance ne peut pas être réservée.
RG6	Une voiture ne peut pas être engagée dans deux réservations actives qui se chevauchent.
RG7	Une demande passe d'abord au statut en_attente , puis peut être acceptée, refusée, annulée ou terminée.
RG8	L'acceptation marque la voiture comme louée ; la finalisation la remet disponible lorsque la location est clôturée.
RG9	Le montant total est recalculé automatiquement à partir de la durée et du prix journalier.
RG10	Un client ne peut consulter ou annuler que ses propres demandes dans les conditions autorisées.
RG11	Les actions importantes sont journalisées et peuvent générer une notification interne.
RG12	La date réelle de retour et une remarque peuvent être enregistrées lors de la clôture.

2.5 Scénarios métiers principaux

2.5.1 Demande de réservation côté client

Le client se connecte, consulte le catalogue, choisit une voiture, saisit les dates et peut vérifier la disponibilité. Le système contrôle les dates, l'état du véhicule et les chevauchements avant d'enregistrer une demande en attente.

2.5.2 Traitement d'une demande côté personnel

Le personnel consulte les réservations, puis accepte, refuse, annule ou termine une location selon son état courant. Les changements de statut mettent à jour le véhicule, l'historique et les notifications.

2.5.3 Pilotage administratif

L'administrateur exploite les listes, les exports, les statistiques, les paiements et la gestion des rôles pour suivre l'activité et maintenir les données.

2.6 Fonctionnalités non retenues

Pour éviter toute confusion, les éléments suivants sont considérés comme des limites de la version actuelle : paiement en ligne, notifications email ou SMS, multi-agence, application mobile native et déploiement cloud opérationnel.

2.7 Sécurité et contrôle des accès

La sécurité repose sur le système d'authentification de Django, les sessions, la protection CSRF et les groupes `Admin` et `Employé`. Les pages internes sont protégées par `login_required`. Les actions sensibles sont réservées à l'administrateur via un contrôle de rôle, tandis que les vues client vérifient la propriété des réservations avant tout affichage ou annulation.

La validation des données est effectuée par les formulaires Django et renforcée dans les vues métier : unicité des champs critiques, cohérence des dates, contrôle des conflits, statut des voitures et transitions autorisées. Les limites restantes concernent surtout la centralisation des validations au niveau modèle, le chargement complet des variables d'environnement et les mesures de production comme HTTPS, serveur web dédié et limitation de débit.

Chapitre 3

Conception et architecture

Ce chapitre présente l’architecture technique, le modèle de données et les principaux schémas UML utilisés pour concevoir l’application.

3.1 Architecture logicielle retenue

L’application suit une architecture monolithique Django organisée autour d’une application métier **core**. La logique est structurée selon le modèle MVT (*Model-View-Template*) : les modèles décrivent les entités métier, les vues portent les traitements et les contrôles d’accès, les templates assurent l’affichage, et les routes relient les URL aux fonctionnalités.

TABLE 3.1 – Choix technologiques essentiels

Technologie	Utilisation	Justification
Django 6.0.5 [1]	Framework principal	ORM, formulaires, authentification, vues et structure MVT intégrés.
SQLite [5]	Base de données	Adaptée au développement local et à la démonstration.
Bootstrap 5 [6]	Interface	Mise en forme responsive et cohérente.
ReportLab [7]	PDF	Génération de factures et de rapports.
Playwright [8]	Tests E2E	Validation de parcours utilisateur dans un navigateur réel.

SQLite reste un choix de développement. Pour une mise en production avec plusieurs utilisateurs, une migration vers PostgreSQL serait plus adaptée.

3.2 Structure logicielle du projet

Listing 3.1 – Structure simplifiée du dépôt

```
pfaKawtar/  
|-- manage.py  
|-- requirements.txt  
|-- location_voiture/  
|   |-- settings.py  
|   |-- urls.py  
|-- core/  
|   |-- models.py  
|   |-- forms.py  
|   |-- views.py  
|   |-- urls.py  
|   |-- templates/core/  
|   |-- tests.py  
|   |-- test_forms.py  
|-- tests_e2e/  
|-- rapport_pfa/
```

3.3 Modèle de données

Le modèle de données repose sur huit entités : **User**, **Client**, **Voiture**, **Reservation**, **Categorie**, **Paiement**, **Historique** et **Notification**. Le lien optionnel entre **Client** et **User** permet de gérer les clients possédant un compte tout en conservant des clients saisis par le personnel.

TABLE 3.2 – Entités métier et responsabilités

Entité	Responsabilité
User	Compte Django utilisé pour l'authentification, les groupes et les notifications.
Client	Informations personnelles du client, avec lien optionnel vers un compte utilisateur.
Voiture	Véhicule du parc : immatriculation, tarif, statut, image et catégorie.
Reservation	Période de location, statut, montant calculé, retour réel et remarque.
Categorie	Classification des véhicules.
Paiement	Paiement manuel associé à une réservation.
Historique	Journalisation des actions importantes.
Notification	Message interne persistant destiné à un utilisateur.

3.4 Diagramme de classes

Le diagramme de classes ci-dessous complète les huit entités du système avec leurs attributs principaux et les cardinalités utilisées dans l'application.

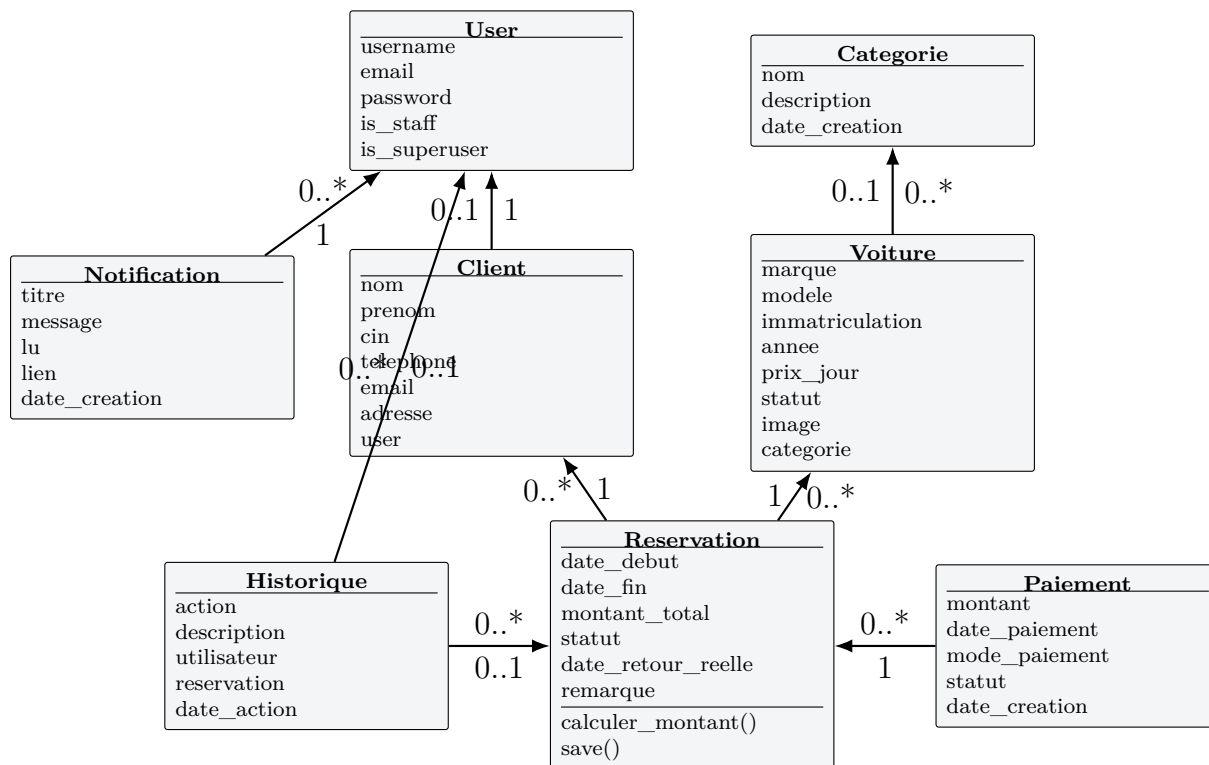


FIGURE 3.1 – Diagramme de classes complet du système

Les associations essentielles sont les suivantes : une réservation concerne exactement un client et une voiture ; une réservation peut recevoir plusieurs paiements ; une voiture peut appartenir à une catégorie ; les actions sont historisées et les notifications sont liées à un utilisateur.

3.5 Modèle logique de données

TABLE 3.3 – Modèle logique de données

Table	Champs principaux
auth_user	id, username, password, email, is_staff, is_superuser
core_client	id, user_id, nom, prenom, cin, telephone, email, adresse
core_voiture	id, categorie_id, marque, modele, immatriculation, annee, prix_jour, statut, image
core_reservation	id, client_id, voiture_id, date_debut, date_fin, montant_total, statut, date_retour_reelle, remarque
core_categorie	id, nom, description, date_creation
core_paiement	id, reservation_id, montant, date_paiement, mode_paiement, statut
core_historique	id, utilisateur_id, reservation_id, action, description, date_action
core_notification	id, utilisateur_id, titre, message, lu, lien, date_creation

3.6 Schémas UML retenus

Les schémas suivants complètent la conception en représentant les interactions, le cycle de vie des réservations et l'organisation technique.

3.6.1 Diagramme de cas d'utilisation

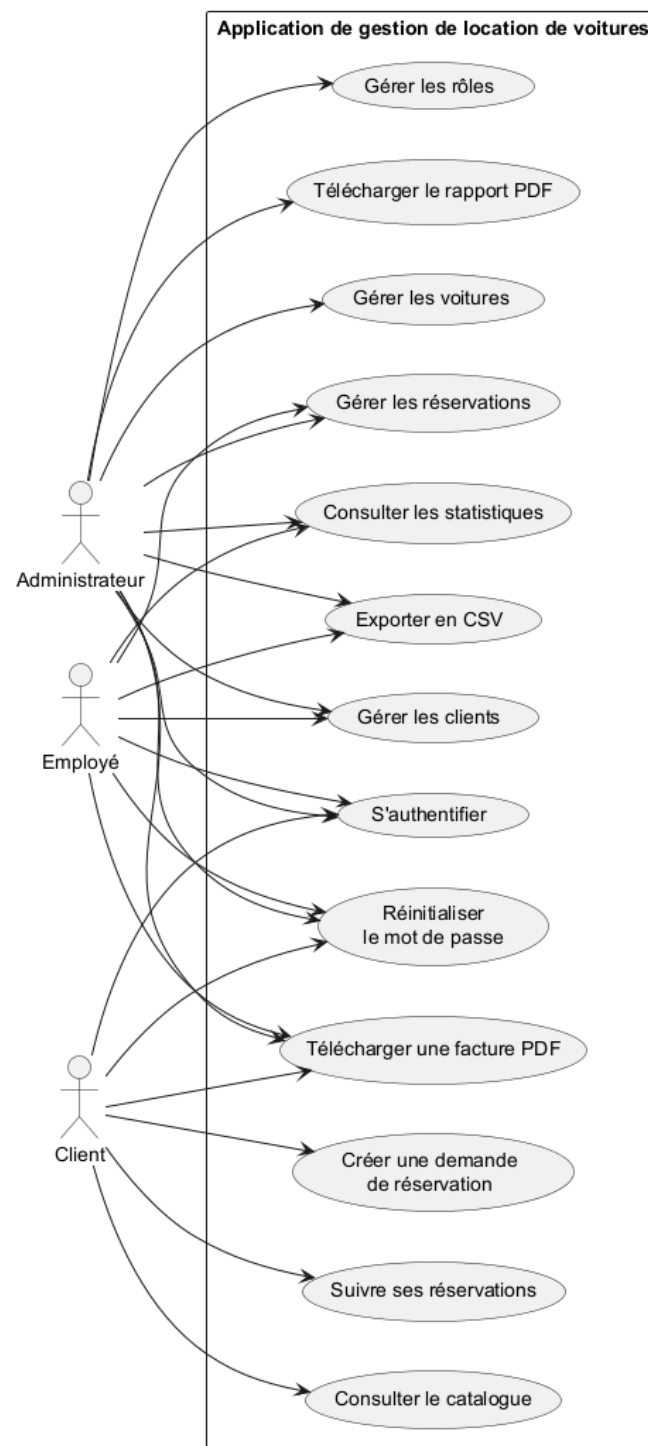


FIGURE 3.2 – Diagramme de cas d'utilisation

3.6.2 Diagramme de séquence : demande de réservation

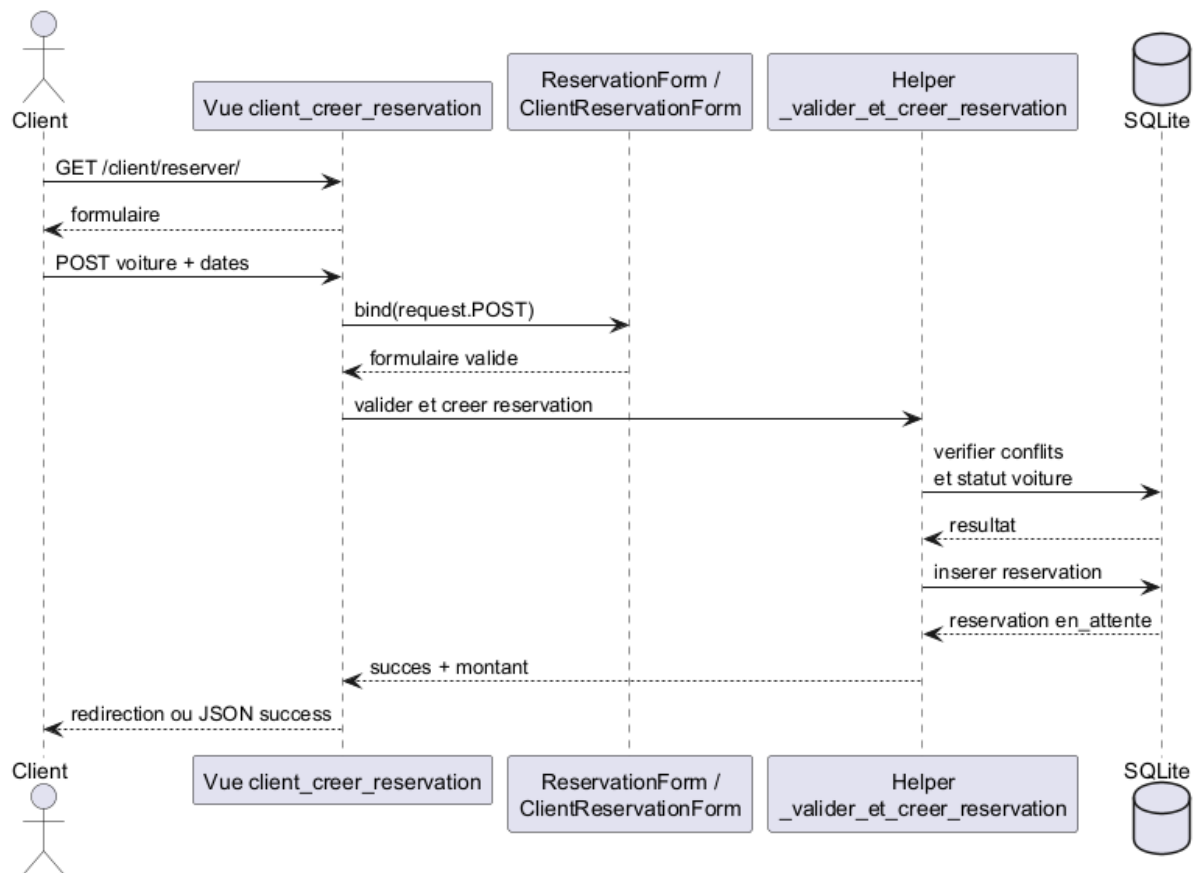


FIGURE 3.3 – Cr  ation d'une demande de r  servation par le client

3.6.3 Diagramme de séquence : traitement par le personnel

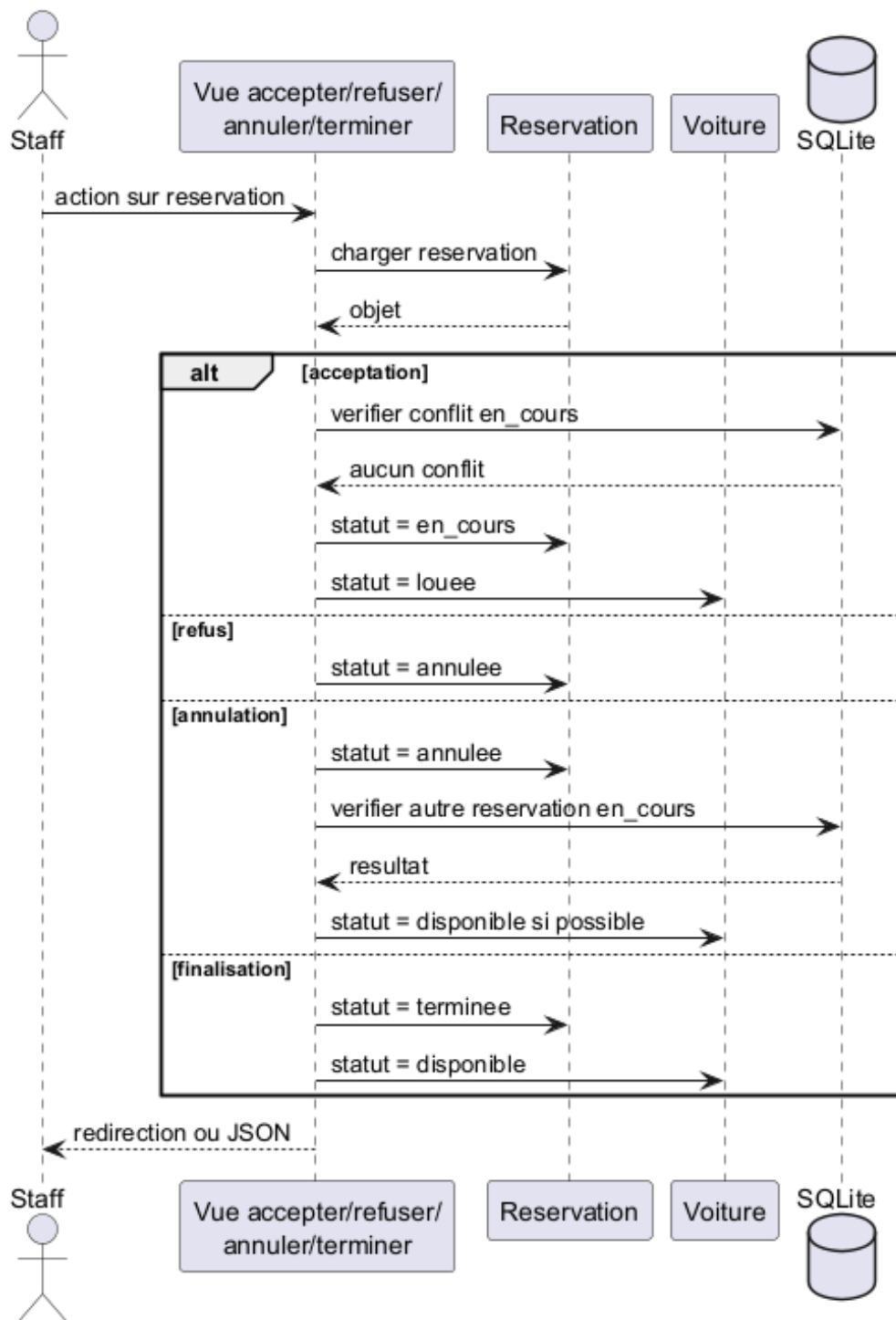


FIGURE 3.4 – Traitement d’une réservation par le personnel

3.6.4 Diagramme d'activité

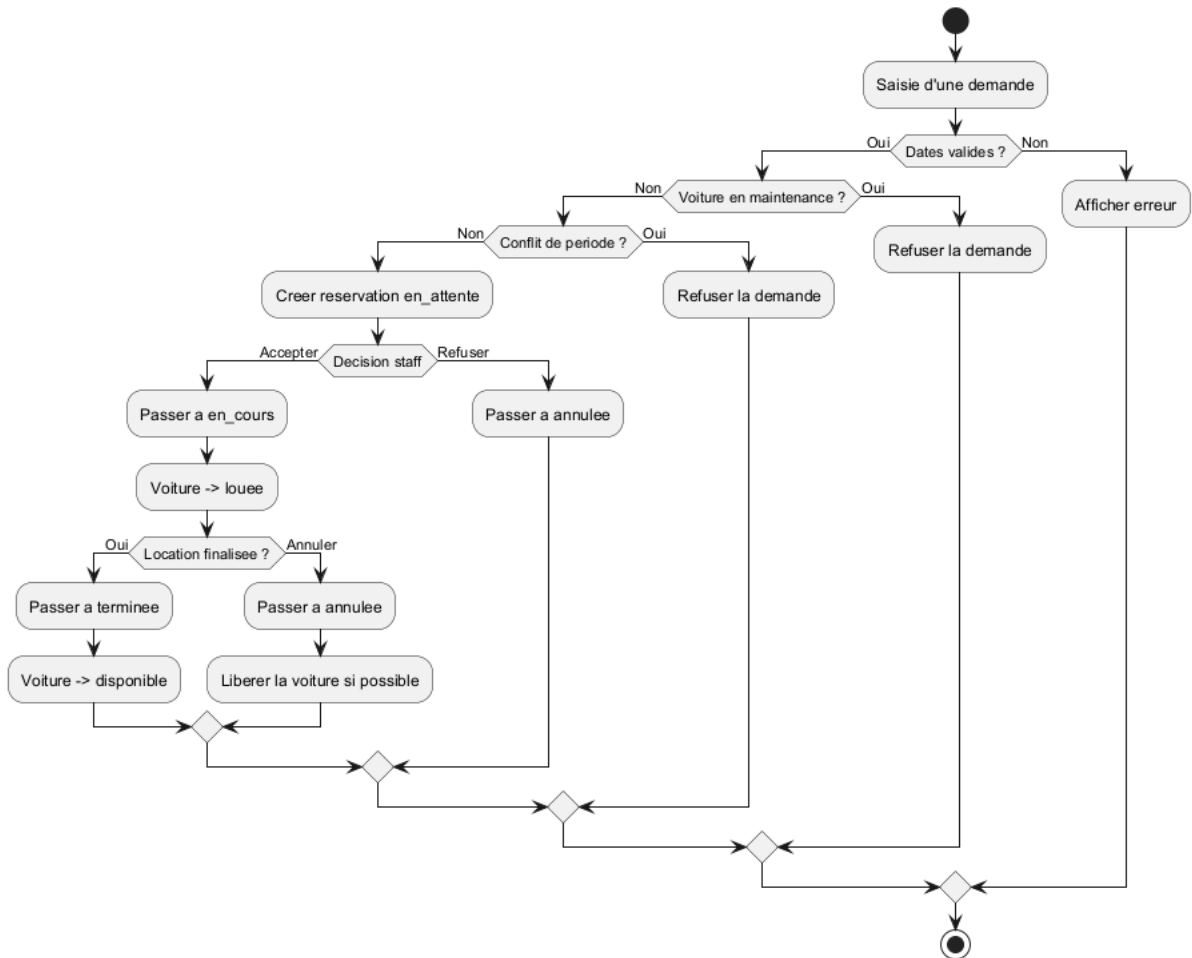


FIGURE 3.5 – Cycle de vie d'une réservation

3.6.5 Diagrammes techniques

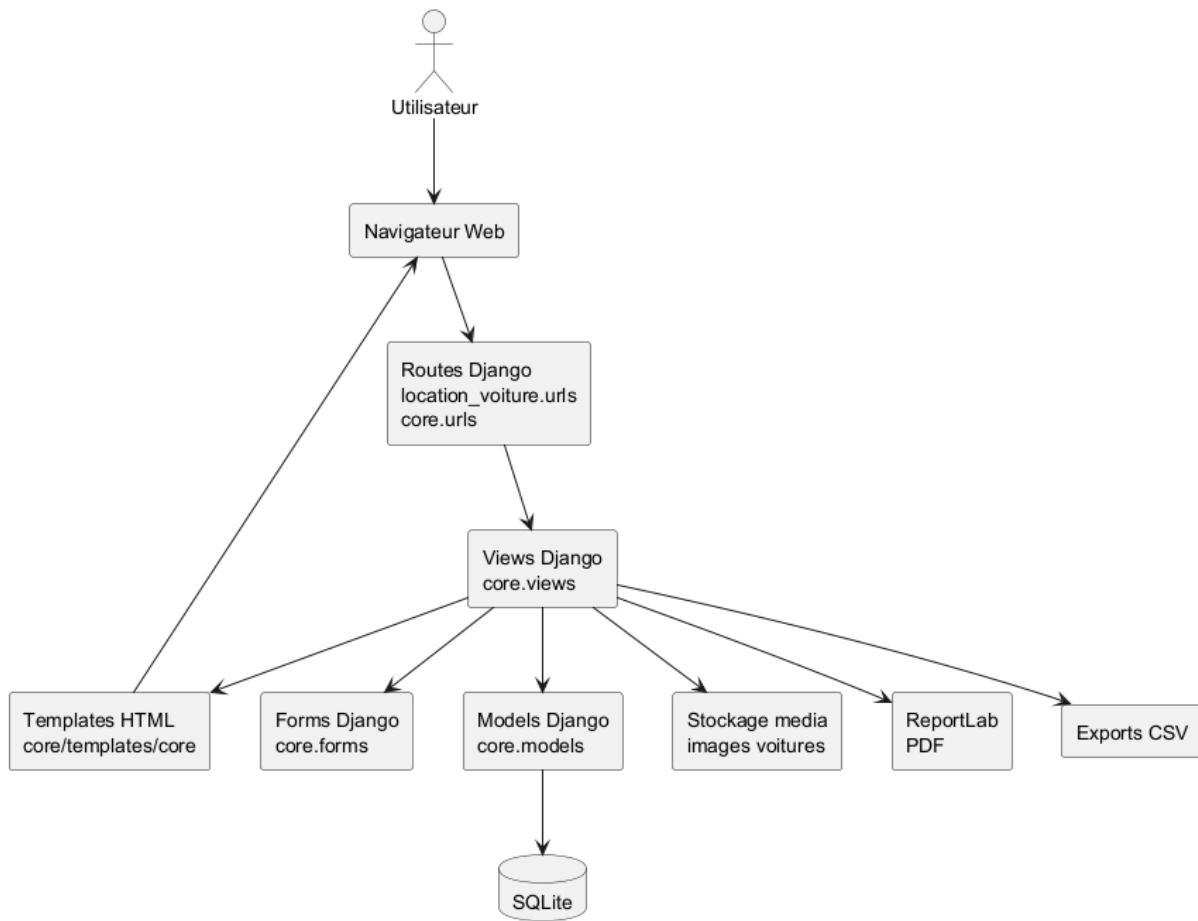


FIGURE 3.6 – Diagramme de composants

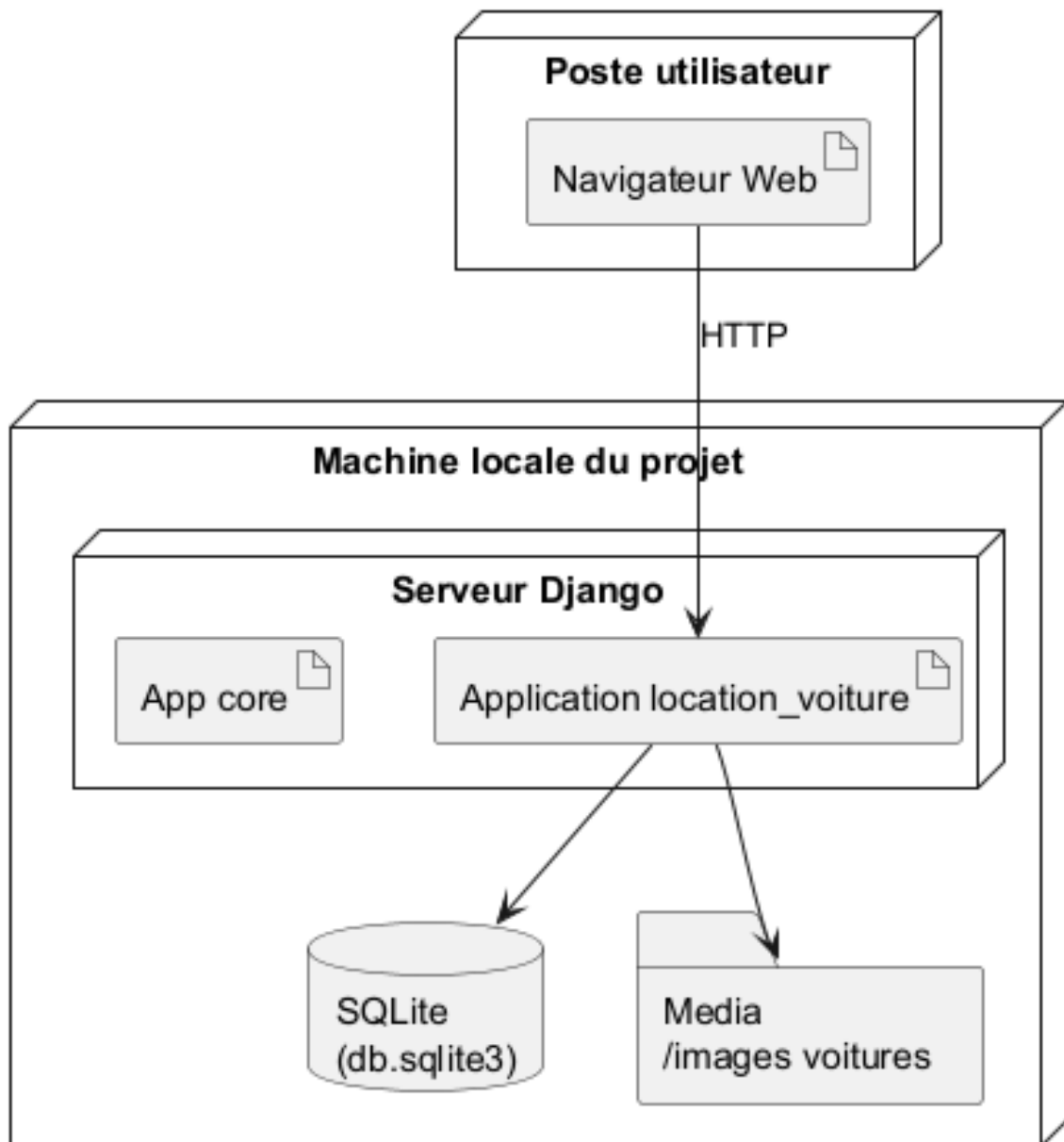


FIGURE 3.7 – Diagramme de déploiement

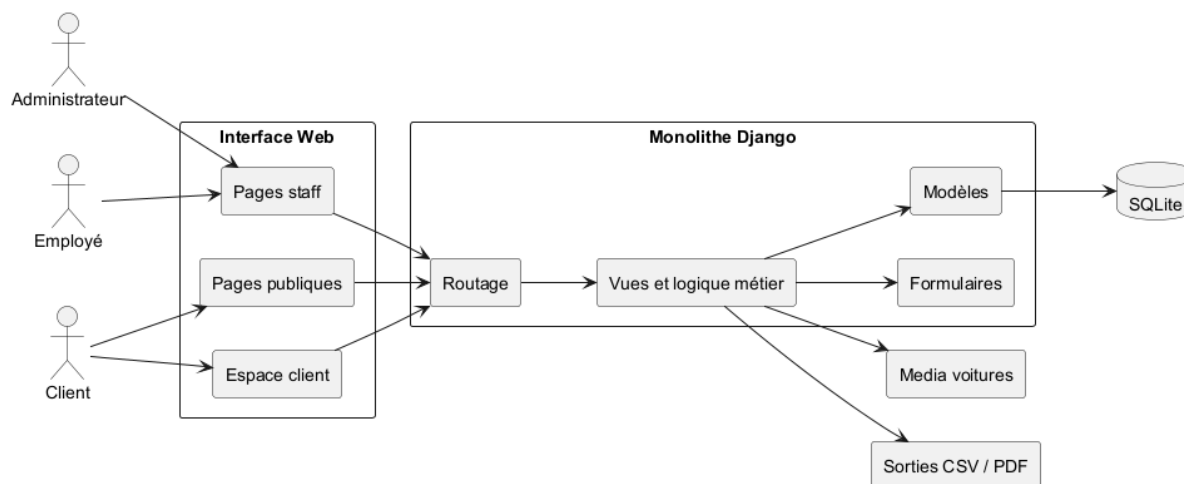


FIGURE 3.8 – Architecture globale du système

3.7 Remarques de conception

L'application privilégie une architecture simple et maintenable, adaptée au périmètre d'un PFA. Les validations métier critiques sont assurées principalement dans les formulaires et les vues. Une amélioration future consisterait à centraliser davantage ces règles au niveau des modèles ou dans une couche service dédiée.

Chapitre 4

Réalisation et fonctionnalités

Ce chapitre présente les fonctionnalités effectivement réalisées. Pour éviter les répétitions, chaque module est décrit selon trois axes : objectif, fonctionnement et résultat obtenu.

4.1 Synthèse des fonctionnalités

TABLE 4.1 – Fonctionnalités réalisées par module

Module	Fonctionnalités principales
Authentification	Connexion, déconnexion, redirection par profil et récupération de mot de passe.
Voitures et catégories	CRUD administrateur, consultation personnel, filtres, recherche, image optionnelle et export CSV.
Clients	Gestion des fiches clients, recherche, détail et historique des réservations.
Réservations	Création, contrôle des conflits, acceptation, refus, annulation, finalisation et facture PDF.
Espace client	Inscription, catalogue, demande de réservation, suivi des demandes et annulation autorisée.
Rôles	Groupes Admin et Employé, restrictions d'accès et gestion du personnel par l'administrateur.
Documents et exports	Exports CSV, facture PDF par réservation et rapport PDF global.
Pilotage	Tableau de bord, statistiques, chiffre d'affaires et indicateurs d'activité.

Paielements	Enregistrement manuel, suivi du statut et consultation par le personnel.
Traçabilité	Historique automatique des actions et notifications internes persistantes.

4.2 Authentification

Objectif : sécuriser l'accès à l'application et orienter chaque utilisateur vers son espace.

Fonctionnement : l'application utilise le système d'authentification de Django. Après connexion, un compte lié à un profil **Client** est redirigé vers l'espace client, tandis que le personnel accède au tableau de bord. Les pages internes sont protégées par `login_required`. La récupération de mot de passe s'appuie sur les vues intégrées de Django.

Résultat obtenu : les accès sont séparés entre administrateur, employé et client, ce qui limite les opérations sensibles aux profils autorisés.

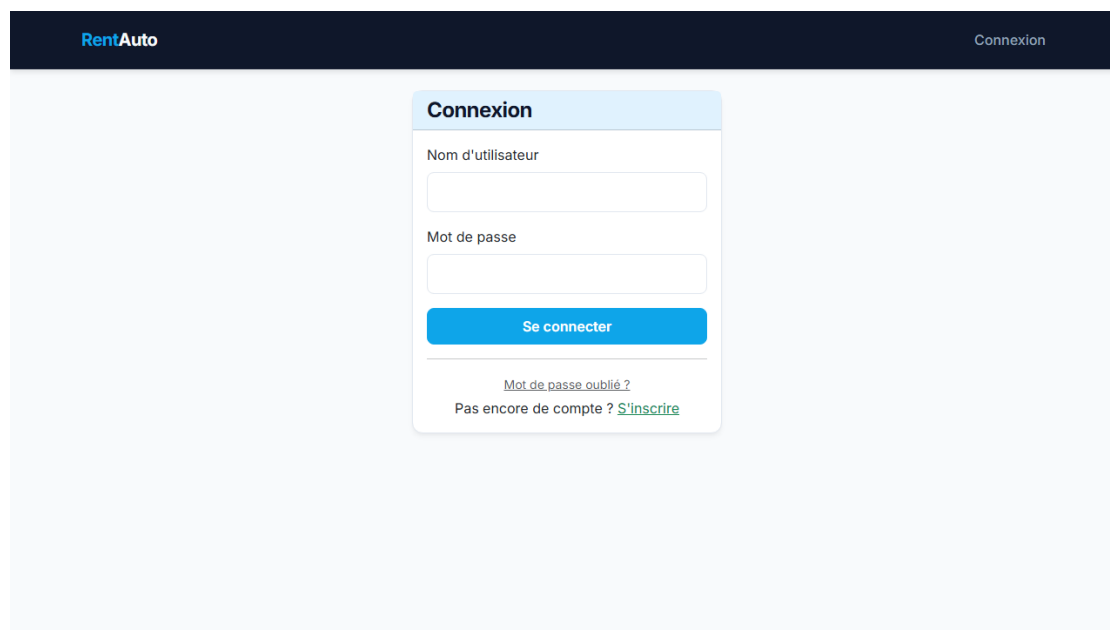


FIGURE 4.1 – Interface de connexion

4.3 Gestion des voitures et des catégories

Objectif : administrer le parc automobile et organiser les véhicules par type.

Fonctionnement : l'administrateur peut ajouter, modifier ou supprimer une voiture avec marque, modèle, immatriculation, année, tarif journalier, statut, catégorie et

image optionnelle. Le personnel consulte la liste, utilise la recherche et filtre par statut ou catégorie. L'immatriculation et le nom de catégorie sont uniques.

Résultat obtenu : le parc automobile est centralisé, filtrable et exportable en CSV. Les véhicules en maintenance sont exclus des demandes de réservation côté client.

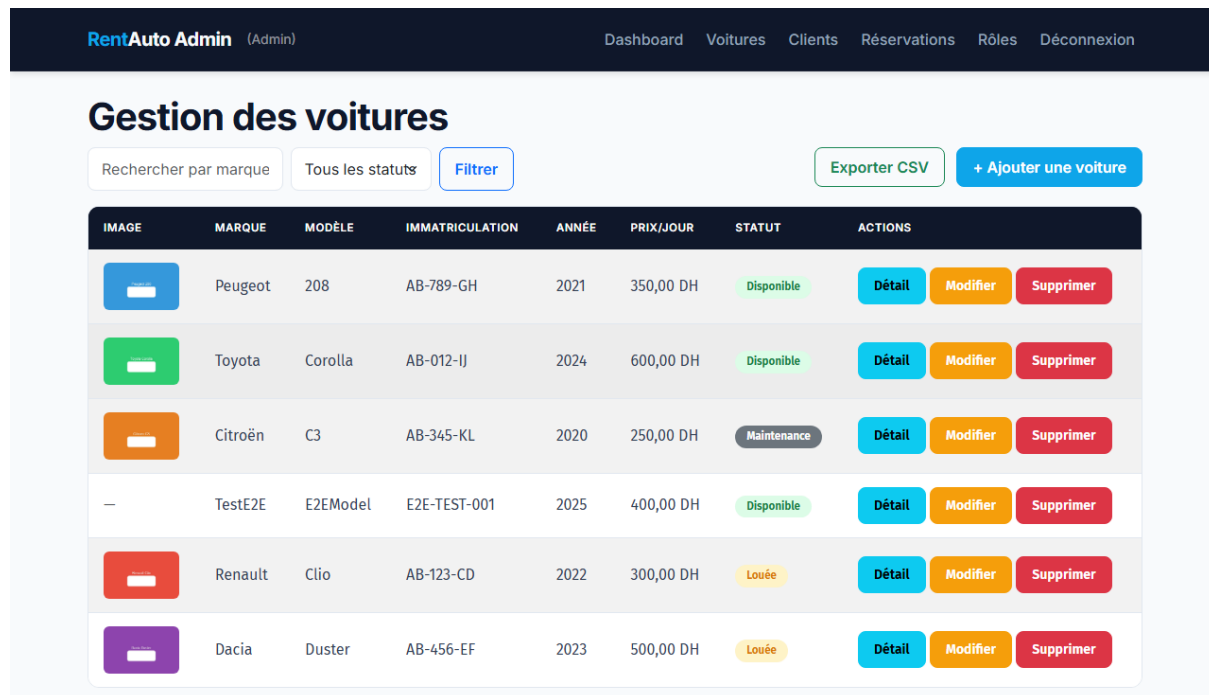


IMAGE	MARQUE	MODÈLE	IMMATRICULATION	ANNÉE	PRIX/JOUR	STATUT	ACTIONS
	Peugeot	208	AB-789-GH	2021	350,00 DH	Disponible	Détail Modifier Supprimer
	Toyota	Corolla	AB-012-IJ	2024	600,00 DH	Disponible	Détail Modifier Supprimer
	Citroën	C3	AB-345-KL	2020	250,00 DH	Maintenance	Détail Modifier Supprimer
—	TestE2E	E2EModel	E2E-TEST-001	2025	400,00 DH	Disponible	Détail Modifier Supprimer
	Renault	Clio	AB-123-CD	2022	300,00 DH	Louée	Détail Modifier Supprimer
	Dacia	Duster	AB-456-EF	2023	500,00 DH	Louée	Détail Modifier Supprimer

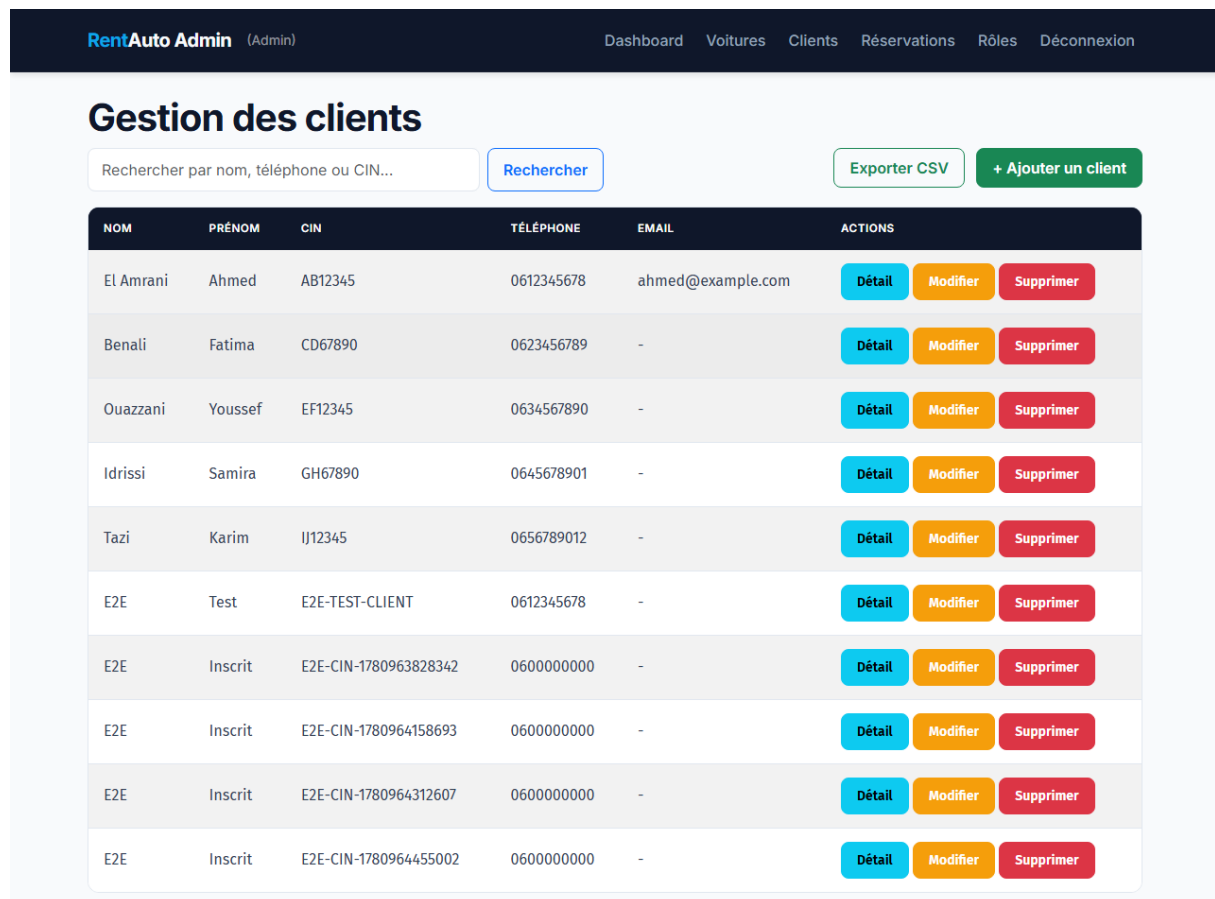
FIGURE 4.2 – Liste des voitures avec recherche et filtres

4.4 Gestion des clients

Objectif : conserver les informations utiles des clients et suivre leur activité.

Fonctionnement : le personnel crée ou consulte les fiches clients. L'administrateur peut les modifier ou les supprimer. Le CIN est unique et un client peut être relié à un compte **User** pour accéder à l'espace client.

Résultat obtenu : les données clients sont regroupées dans une base unique, avec accès rapide à l'historique des réservations.



RentAuto Admin (Admin) Dashboard Voitures Clients Réservations Rôles Déconnexion

Gestion des clients

Rechercher par nom, téléphone ou CIN... [Rechercher](#) [Exporter CSV](#) [+ Ajouter un client](#)

NOM	PRÉNOM	CIN	TÉLÉPHONE	EMAIL	ACTIONS
El Amrani	Ahmed	AB12345	0612345678	ahmed@example.com	Détail Modifier Supprimer
Benali	Fatima	CD67890	0623456789	-	Détail Modifier Supprimer
Ouazzani	Youssef	EF12345	0634567890	-	Détail Modifier Supprimer
Idrissi	Samira	GH67890	0645678901	-	Détail Modifier Supprimer
Tazi	Karim	IJ12345	0656789012	-	Détail Modifier Supprimer
E2E	Test	E2E-TEST-CLIENT	0612345678	-	Détail Modifier Supprimer
E2E	Inscrit	E2E-CIN-1780963828342	0600000000	-	Détail Modifier Supprimer
E2E	Inscrit	E2E-CIN-1780964158693	0600000000	-	Détail Modifier Supprimer
E2E	Inscrit	E2E-CIN-1780964312607	0600000000	-	Détail Modifier Supprimer
E2E	Inscrit	E2E-CIN-1780964455002	0600000000	-	Détail Modifier Supprimer

FIGURE 4.3 – Liste des clients

4.5 Réservations

Objectif : gérer le cycle complet d’une location, depuis la demande jusqu’à la clôture.

Fonctionnement : une réservation contient un client, une voiture, une période, un statut et un montant calculé automatiquement. Les vues contrôlent les dates, les conflits de réservation et le statut du véhicule. Une demande acceptée passe en cours et marque la voiture comme louée ; une location terminée remet la voiture disponible et peut enregistrer une date réelle de retour.

Résultat obtenu : les erreurs principales de gestion manuelle sont réduites : double réservation, montant incorrect, absence de suivi d’état ou retour non tracé.

RentAuto Admin (Admin)

DashboardVoituresClientsRéservationsRôlesDéconnexion

Gestion des réservations

Tous les statuts

Filtrer

Exporter CSV

+ Nouvelle réservation

CLIENT	VOITURE	DATE DÉBUT	DATE FIN	MONTANT	STATUT	ACTIONS
Ouazzani Youssef	 Peugeot 208	09/06/2026	13/06/2026	1400,00 DH	Terminée	Détail Modifier PDF
Benali Fatima	 Dacia Duster	14/06/2026	17/06/2026	1500,00 DH	En cours	Détail Modifier Terminer Annuler
El Amrani Ahmed	 Renault Clio	12/06/2026	16/06/2026	1200,00 DH	En cours	Détail Modifier Terminer Annuler
Ouazzani Youssef	 Peugeot 208	04/06/2026	08/06/2026	1400,00 DH	Terminée	Détail Modifier PDF
Benali Fatima	 Dacia Duster	09/06/2026	12/06/2026	1500,00 DH	En cours	Détail Modifier Terminer Annuler
El Amrani Ahmed	 Renault Clio	07/06/2026	11/06/2026	1200,00 DH	En cours	Détail Modifier Terminer Annuler
Ouazzani Youssef	 Peugeot 208	04/06/2026	08/06/2026	1400,00 DH	Terminée	Détail Modifier PDF
Ouazzani Youssef	 Peugeot 208	03/06/2026	07/06/2026	1400,00 DH	Terminée	Détail Modifier PDF
Ouazzani Youssef	 Peugeot 208	03/06/2026	07/06/2026	1400,00 DH	Terminée	Détail Modifier PDF
Ouazzani Youssef	 Peugeot 208	03/06/2026	07/06/2026	1400,00 DH	Terminée	Détail Modifier PDF

FIGURE 4.4 – Liste des réservations et actions de traitement

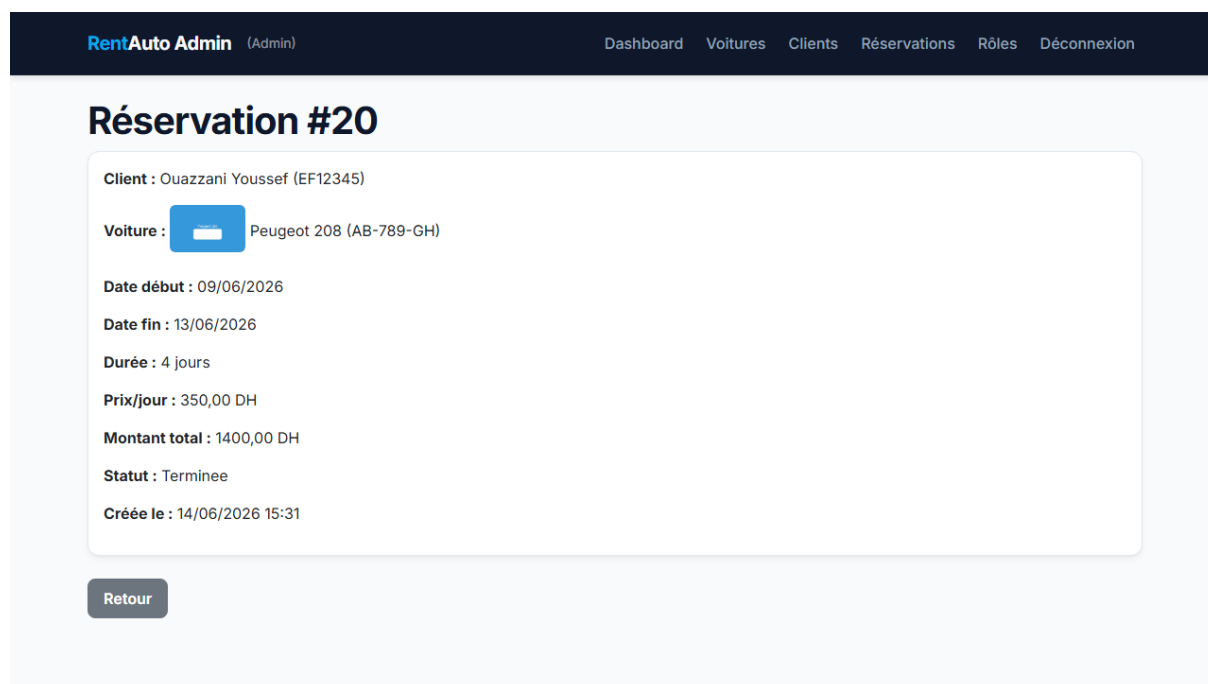


FIGURE 4.5 – Détail d’une réservation avec actions, paiements et historique

4.6 Espace client

Objectif : permettre au client de consulter le catalogue et de déposer ses demandes sans passer par le personnel.

Fonctionnement : l’inscription crée simultanément un compte Django et un profil **Client**. Une fois connecté, le client consulte les voitures, vérifie la disponibilité par AJAX, crée une demande en attente et suit ses réservations. Il ne peut voir ou annuler que ses propres demandes.

Résultat obtenu : le client dispose d’un parcours autonome, tandis que le personnel conserve le contrôle de la validation finale.

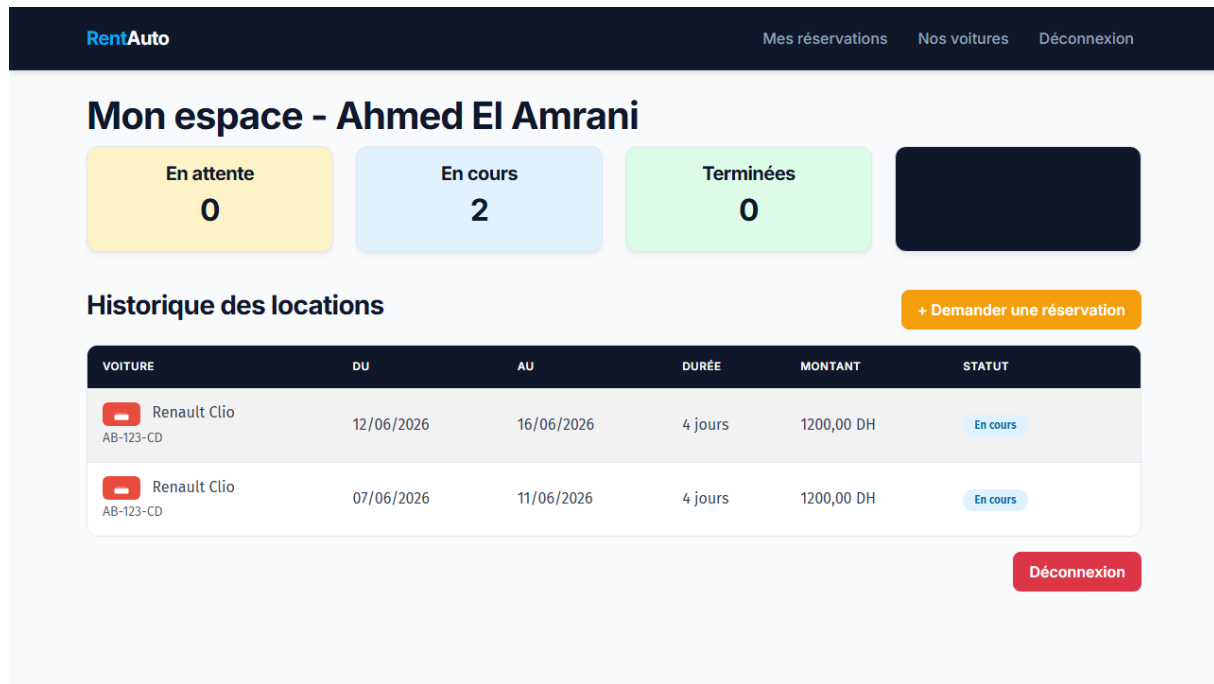


FIGURE 4.6 – Tableau de bord client

RentAuto Mes réservations Nos voitures Déconnexion

Nos voitures

Rechercher par marque, modèle... [Rechercher](#) [+ Demander une réservation](#)

Peugeot 208

Peugeot 208
AB-789-GH — 2021
350,00 DH /jour
Disponible
[Voir détails](#)

Toyota Corolla

Toyota Corolla
AB-012-IJ — 2024
600,00 DH /jour
Disponible
[Voir détails](#)

Pas d'image

TestE2E E2EModel
E2E-TEST-001 — 2025
400,00 DH /jour
Disponible
[Voir détails](#)

Renault Clio

Renault Clio
AB-123-CD — 2022
300,00 DH /jour
Louée
[Voir détails](#)

Dacia Duster

Dacia Duster
AB-456-EF — 2023
500,00 DH /jour
Louée
[Voir détails](#)

[Retour au dashboard](#)

FIGURE 4.7 – Catalogue des voitures côté client

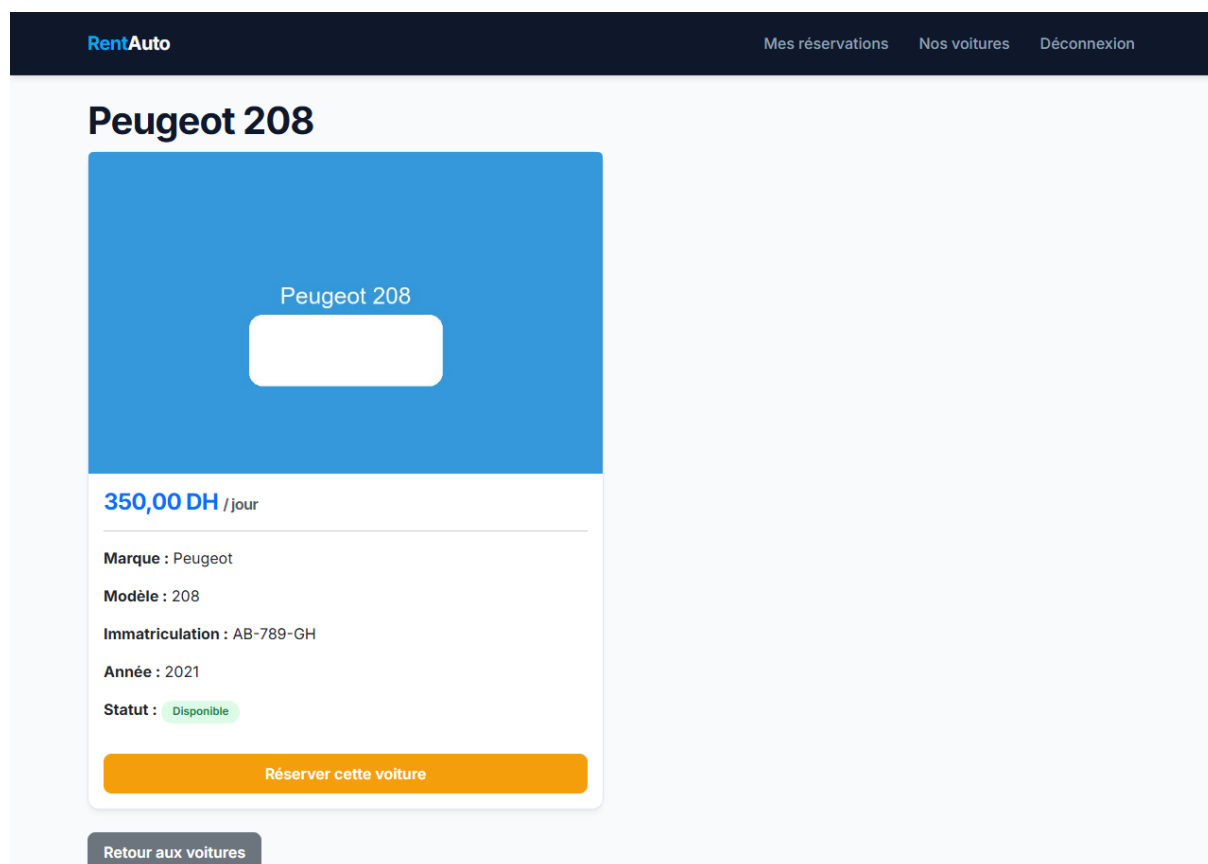


FIGURE 4.8 – Vérification de disponibilité côté client

4.7 Gestion des rôles

Objectif : distinguer les responsabilités entre administrateur et employé.

Fonctionnement : la séparation repose sur les groupes Django **Admin** et **Employé**. Les vues sensibles utilisent un contrôle de rôle. L'administrateur peut créer des comptes du personnel, modifier les groupes et supprimer les comptes non superutilisateurs.

Résultat obtenu : les actions critiques comme la gestion des rôles, la modification du parc ou l'enregistrement des paiements sont réservées aux profils autorisés.

Gestion des utilisateurs

Ajouter un utilisateur

Nom d'utilisateur Mot de passe Rôle

-- Aucun -- Ajouter

UTILISATEUR	RÔLE ACTUEL	CHANGER LE RÔLE	SUPPRIMER
admin	Admin	Admin Appliquer	Supprimer
employe	Employé	Employé Appliquer	Supprimer

Retour

FIGURE 4.9 – Gestion des rôles du personnel

4.8 Exports PDF et CSV

Objectif : produire des documents exploitables à partir des données de l'application.

Fonctionnement : les vues d'export retournent des fichiers CSV pour les voitures, les clients et les réservations. Les factures PDF et le rapport global sont générés avec ReportLab à partir des informations stockées en base.

Résultat obtenu : l'application fournit des documents de suivi et de facturation utiles pour la démonstration et pour une exploitation administrative simple.

RentAuto Admin (Admin)							Dashboard Voitures Clients Réservations Rôles Déconnexion		
Gestion des réservations							Tous les statuts Filtrer Exporter CSV + Nouvelle réservation		
CLIENT	VOITURE	DATE DÉBUT	DATE FIN	MONTANT	STATUT	ACTIONS			
Ouazzani Youssef	 Peugeot 208	09/06/2026	13/06/2026	1400,00 DH	Terminée	Détail Modifier PDF			
Benali Fatima	 Dacia Duster	14/06/2026	17/06/2026	1500,00 DH	En cours	Détail Modifier Terminer Annuler			
El Amrani Ahmed	 Renault Clio	12/06/2026	16/06/2026	1200,00 DH	En cours	Détail Modifier Terminer Annuler			
Ouazzani Youssef	 Peugeot 208	04/06/2026	08/06/2026	1400,00 DH	Terminée	Détail Modifier PDF			
Benali Fatima	 Dacia Duster	09/06/2026	12/06/2026	1500,00 DH	En cours	Détail Modifier Terminer Annuler			
El Amrani Ahmed	 Renault Clio	07/06/2026	11/06/2026	1200,00 DH	En cours	Détail Modifier Terminer Annuler			
Ouazzani Youssef	 Peugeot 208	04/06/2026	08/06/2026	1400,00 DH	Terminée	Détail Modifier PDF			
Ouazzani Youssef	 Peugeot 208	03/06/2026	07/06/2026	1400,00 DH	Terminée	Détail Modifier PDF			
Ouazzani Youssef	 Peugeot 208	03/06/2026	07/06/2026	1400,00 DH	Terminée	Détail Modifier PDF			
Ouazzani Youssef	 Peugeot 208	03/06/2026	07/06/2026	1400,00 DH	Terminée	Détail Modifier PDF			

FIGURE 4.10 – Facture PDF générée pour une réservation

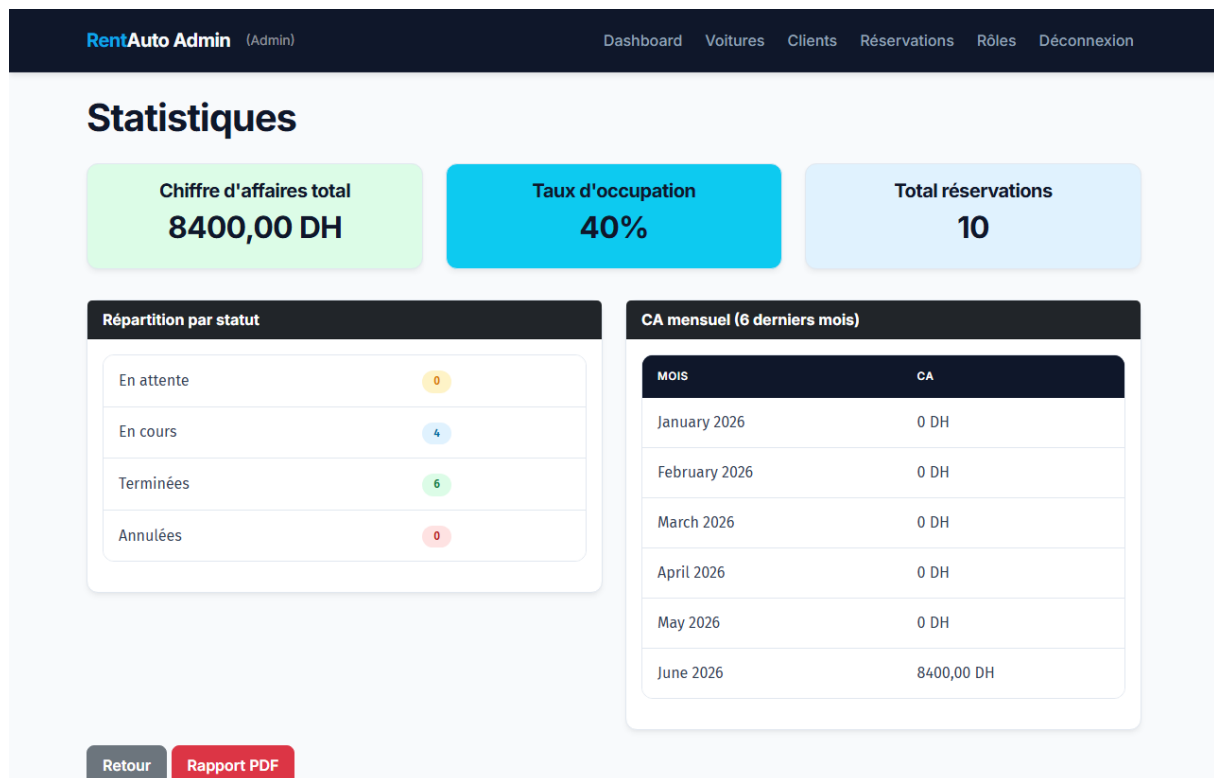


FIGURE 4.11 – Rapport PDF global

4.9 Statistiques

Objectif : offrir une vision synthétique de l'activité de l'agence.

Fonctionnement : la page statistiques s'appuie sur les agrégations Django pour calculer les indicateurs : nombre de voitures, réservations, paiements, chiffre d'affaires, taux d'occupation et évolution mensuelle.

Résultat obtenu : le personnel dispose d'indicateurs simples pour suivre l'activité sans manipuler directement la base de données.

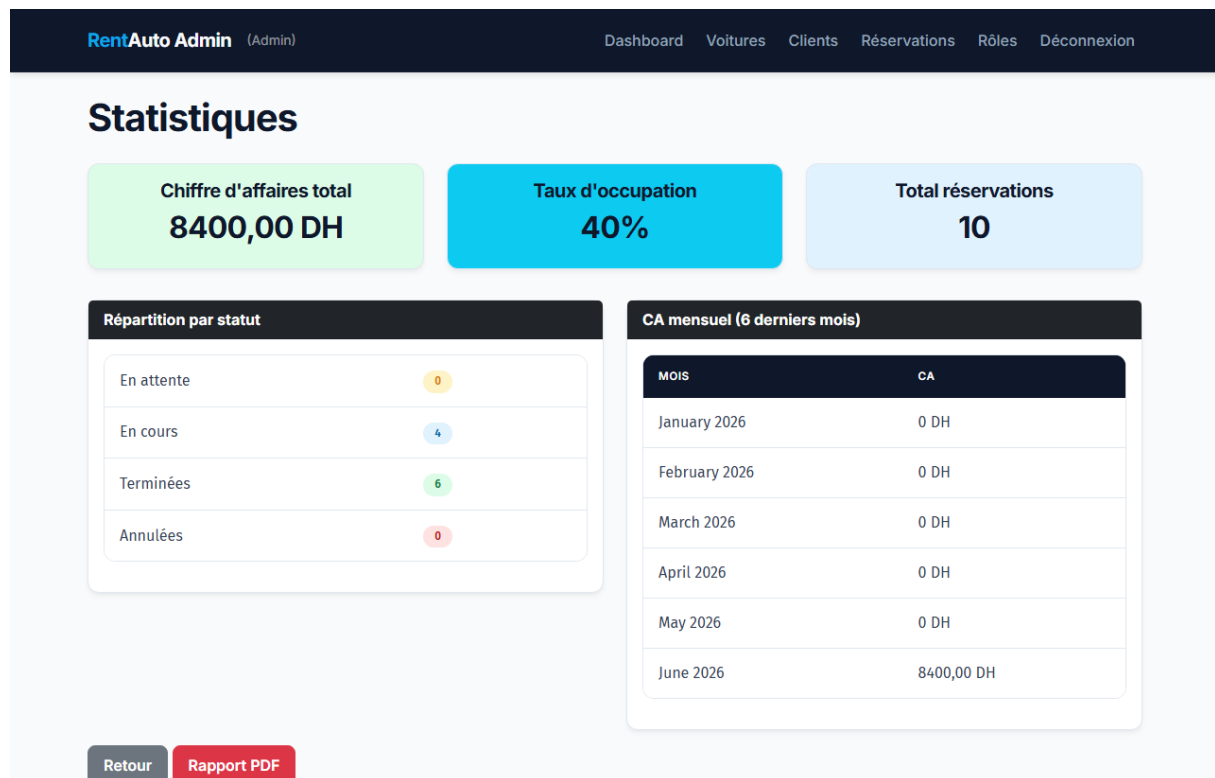


FIGURE 4.12 – Page des statistiques

4.10 Paiements

Objectif : suivre les règlements associés aux réservations.

Fonctionnement : le paiement est enregistré manuellement par l'administrateur avec montant, date, mode et statut. Il reste lié à une réservation et peut être consulté par le personnel.

Résultat obtenu : le projet couvre le suivi interne des paiements, sans prétendre intégrer un paiement en ligne.

4.11 Historique et notifications

Objectif : améliorer la traçabilité et informer les utilisateurs des actions importantes.

Fonctionnement : un helper crée des entrées dans **Historique** lors des événements métier : création, validation, refus, annulation, paiement ou retour. Les notifications internes sont liées à un utilisateur et peuvent être marquées comme lues.

Résultat obtenu : les principales actions sont traçables, et les utilisateurs disposent d'un retour interne sur les événements qui les concernent.

4.12 Bilan de réalisation

La réalisation couvre les besoins essentiels du cahier des charges : gestion du parc, clients, réservations, espace client, rôles, documents, statistiques, paiements, historique et notifications. Les détails techniques secondaires, comme les routes principales et les extraits de code métier, sont placés en annexes afin de conserver un corps de rapport plus lisible.

Chapitre 5

Tests et validation

La validation du projet repose sur deux niveaux complémentaires : 222 tests Django et 35 tests end-to-end Playwright. Les tests ne vérifient pas seulement l’affichage des pages ; ils contrôlent surtout les règles métier critiques, les droits d’accès et les parcours utilisateur.

5.1 Stratégie de test

- **Tests Django** : 222 tests unitaires et fonctionnels couvrant les modèles, formulaires, vues, règles métier, exports, PDF, rôles, espace client, paiements, historique et notifications.
- **Tests Playwright** [8] : 35 scénarios exécutés dans un navigateur réel pour valider les principaux parcours d’utilisation.

Les cas les plus sensibles concernent les dates invalides, les conflits de réservation, les voitures en maintenance, les accès non autorisés, la propriété des réservations côté client et les changements de statut.

5.2 Résultat de la suite Django

La commande de validation utilisée est la suivante :

```
python manage.py test core -v2
```

Les 222 tests Django ont été exécutés avec succès. Ce résultat confirme que les fonctionnalités principales respectent les règles métier définies dans l’analyse des besoins.

5.3 Couverture fonctionnelle

TABLE 5.1 – Couverture synthétique des tests Django

Catégorie	Éléments vérifiés
Formulaires	Champs obligatoires, unicité, dates, inscription client, paiements et profil.
Modèles	Représentations, valeurs par défaut, calcul du montant, catégories, historique et notifications.
Vues métier	CRUD voitures/clients, réservations, exports CSV, PDF, statistiques et erreurs.
Règles critiques	Conflits, maintenance, transitions de statut, retour réel et annulation côté client.
Sécurité	Authentification, RBAC, restrictions d'accès et isolation des données client.

5.4 Exemples de tests représentatifs

TABLE 5.2 – Exemples de tests représentatifs

Exemple	Vérification	Intérêt métier
Unicité immatriculation	Deux voitures ne peuvent pas partager la même immatriculation.	Évite les doublons dans le parc automobile.
Conflit de réservation	Une voiture déjà engagée sur une période active ne peut pas être réservée à nouveau.	Empêche la double location.
Acceptation AJAX	L'acceptation d'une demande retourne un JSON cohérent et met à jour le statut.	Valide le traitement sans rechargement complet.
Annulation client	Un client ne peut annuler que ses propres demandes autorisées.	Protège les données des autres clients.
Accès RBAC	Un employé ne peut pas accéder à la gestion des rôles.	Garantit la séparation des responsabilités.

5.5 Tests end-to-end Playwright

Les 35 tests Playwright sont regroupés dans `tests_e2e/app.spec.js`. Ils simulent des actions réelles dans le navigateur : connexion, navigation, consultation des listes, création de données, traitement d'une réservation, téléchargement de documents, accès à l'espace client et vérification des restrictions de rôle.

TABLE 5.3 – Couverture synthétique des tests Playwright

Parcours	Scénarios validés
Authentification	Accueil, connexion admin/client, mauvais identifiants, redirection et déconnexion.
Back-office	Navigation, listes voitures/clients/réservations, filtres, détails et formulaires.
Documents	Exports CSV et facture PDF téléchargeable.
Espace client	Inscription, tableau de bord, catalogue et demande de réservation.
Rôles et erreurs	Accès admin autorisé, employé bloqué et page 404 fonctionnelle.

5.6 Recette fonctionnelle

La recette manuelle reprend le parcours principal : connexion administrateur, ajout d’une voiture, création ou inscription d’un client, demande de réservation, vérification de disponibilité, acceptation par le personnel, changement de statut du véhicule, finalisation, génération de facture PDF, consultation des statistiques et export CSV. Ces étapes sont couvertes par les tests automatisés.

5.7 Avertissements et limites

Deux avertissements techniques ont été identifiés lors des tests : certaines listes paginées gagneraient à être ordonnées explicitement, et quelques objets de test utilisent des dates naïves alors que `USE_TZ = True`. Ces points ne bloquent pas l’application, mais ils constituent des améliorations utiles.

Les limites principales restent les suivantes : validation métier encore partagée entre formulaires et vues, absence de chargement complet des variables `.env`, SQLite utilisé pour la démonstration, absence de paiement en ligne et notifications limitées à l’application.

5.8 Commandes de validation

Listing 5.1 – Commandes de test

```
python manage.py test core -v2
npx playwright test
```

La couverture obtenue constitue un socle fiable pour faire évoluer le projet tout en limitant les régressions.

Chapitre 6

Conclusion et perspectives

Ce projet de fin d'année a permis de concevoir, développer et valider une application web de gestion de location de voitures avec Django. La solution obtenue centralise les véhicules, les clients, les réservations, les paiements, les documents et les indicateurs d'activité dans une application unique.

Les objectifs essentiels ont été atteints. L'application propose une authentification avec séparation des profils, une gestion du parc automobile, un suivi des clients, un cycle complet de réservation, un espace client, des exports CSV, des factures PDF, un rapport global, des statistiques, une gestion des rôles, un historique et des notifications internes. Les règles métier importantes sont prises en compte : contrôle des conflits de réservation, refus des voitures en maintenance, calcul automatique du montant, transitions de statut et vérification de la propriété des réservations côté client.

Le projet apporte aussi une expérience technique complète : modélisation UML, architecture MVT, utilisation de l'ORM Django, formulaires, vues protégées, génération PDF, requêtes AJAX et tests automatisés. La validation repose sur **222 tests Django** et **35 tests Playwright**, ce qui renforce la fiabilité de la solution et facilite ses évolutions futures.

Certaines limites doivent toutefois être reconnues. La base SQLite convient au développement, mais une production multi-utilisateur nécessiterait PostgreSQL. Les validations métier sont encore principalement portées par les formulaires et les vues ; elles pourraient être davantage centralisées. Le projet n'est pas déployé sur un serveur cloud, le paiement en ligne n'est pas implémenté et les notifications restent internes à l'application.

Les perspectives les plus réalistes sont donc la migration vers PostgreSQL, le déploiement sécurisé avec variables d'environnement, l'intégration d'un paiement en ligne, l'envoi de notifications email ou SMS, l'ajout d'un calendrier visuel des disponibilités, l'optimisation des images et le renforcement des validations métier. Ces améliorations prolongeraient naturellement le socle réalisé sans modifier la logique générale de l'application.

En conclusion, le projet constitue une base fonctionnelle, démontrable et extensible pour la gestion d'une agence de location de voitures. Il répond au besoin principal de centralisation et montre une démarche complète allant de l'analyse à la validation.

Bibliographie

Bibliographie

- [1] Django Software Foundation. *Django 6.0 documentation*. Disponible sur : <https://docs.djangoproject.com/>
- [2] Django Software Foundation. *Models – Django documentation*. Disponible sur : <https://docs.djangoproject.com/en/6.0/topics/db/models/>
- [3] Django Software Foundation. *Forms – Django documentation*. Disponible sur : <https://docs.djangoproject.com/en/6.0/topics/forms/>
- [4] Django Software Foundation. *Authentication in Django – Django documentation*. Disponible sur : <https://docs.djangoproject.com/en/6.0/topics/auth/>
- [5] SQLite Consortium. *SQLite documentation*. Disponible sur : <https://www.sqlite.org/docs.html>
- [6] Bootstrap Team. *Bootstrap 5 documentation*. Disponible sur : <https://getbootstrap.com/docs/5.3/>
- [7] ReportLab. *ReportLab User Guide*. Disponible sur : <https://www.reportlab.com/documentation/>
- [8] Microsoft. *Playwright documentation – end-to-end testing for modern web apps*. Disponible sur : <https://playwright.dev/python/>
- [9] OWASP Foundation. *OWASP Top Ten – Web Application Security Risks*. Disponible sur : <https://owasp.org/www-project-top-ten/>
- [10] OWASP Foundation. *OWASP Web Security Testing Guide*. Disponible sur : <https://owasp.org/www-project-web-security-testing-guide/>
- [11] Object Management Group. *Unified Modeling Language (UML) specification*. Disponible sur : <https://www.omg.org/spec/UML/>
- [12] Booch, G., Rumbaugh, J., Jacobson, I. *The Unified Modeling Language User Guide*. 2^e édition, Addison-Wesley, 2005.
- [13] Python Software Foundation. *Python 3 documentation*. Disponible sur : <https://docs.python.org/3/>

Annexe A

Annexes

A.1 Commandes d'installation et d'exécution

Les commandes suivantes permettent de mettre en place l'environnement de développement :

Listing A.1 – Installation et exécution de l'application

```
# Creation de l'environnement virtuel
python -m venv venv
source venv/bin/activate # Linux/Mac
venv\Scripts\activate # Windows

# Installation des dependances
pip install -r requirements.txt

# Application des migrations
python manage.py migrate

# Creation des donnees de demonstration
python manage.py seed_data

# Lancement du serveur de developpement
python manage.py runserver
```

A.2 Identifiants de démonstration

Après exécution de la commande `seed_data`, les comptes suivants sont disponibles :

TABLE A.1 – Comptes de démonstration

Profil	Identifiant	Mot de passe
Administrateur	admin	admin1234
Employé	employe	employe1234
Client	client	client1234

A.3 Extrait des routes principales

Listing A.2 – Routes principales de l'application

```

path("", views.accueil, name="accueil")
path("login/", views.login_view, name="login")
path("logout/", views.logout_view, name="logout")
path("dashboard/", views.dashboard, name="dashboard")
path("password-reset/", views.password_reset_view, name="
    password_reset")
path("voitures/", views.liste_voitures, name="liste_voitures")
path("voitures/ajouter/", views.ajouter_voiture, name="
    ajouter_voiture")
path("clients/", views.liste_clients, name="liste_clients")
path("reservations/", views.liste_reservations, name="
    liste_reservations")
path("client/dashboard/", views.client_dashboard, name="
    client_dashboard")
path("client/voitures/", views.client_voitures, name="
    client_voitures")
path("client/reserver/", views.client_creer_reservation, name="
    client_creer_reservation")
path("roles/", views.gestion_roles, name="gestion_roles")
path("statistiques/", views.statistiques, name="statistiques")
path("historique/", views.historique, name="historique")
path("notifications/", views.liste_notifications, name="
    liste_notifications")
path("rapport/pdf/", views.rapport_pdf, name="rapport_pdf")

```

A.4 Extrait de calcul métier

Listing A.3 – Calcul automatique du montant total

```

def calculer_montant(self):

```

```
nombre_jours = (self.date_fin - self.date_debut).days
if nombre_jours <= 0:
    nombre_jours = 1
return nombre_jours * self.voiture.prix_jour
```

A.5 Exemple de contrôle de conflit

Listing A.4 – Detection des chevauchements de reservation

```
conflit = Reservation.objects.filter(
    voiture=reservation.voiture,
    statut__in=["en_attente", "en_cours"],
    date_debut__lt=reservation.date_fin,
    date_fin__gt=reservation.date_debut,
)
```

A.6 Extrait de journalisation

Listing A.5 – Fonction d'enregistrement dans l'historique

```
def enregistrer_historique(action, description,
                           utilisateur=None, reservation=None):
    Historique.objects.create(
        action=action,
        description=description,
        utilisateur=utilisateur,
        reservation=reservation,
    )
```

A.7 Structure du fichier requirements.txt

Listing A.6 – Dependances du projet

```
Django>=6.0,<6.1
reportlab>=4.0
Pillow>=11.0
```


A.8 Variables d'environnement

Le fichier `.env.example` fournit le modèle de configuration suivant :

Listing A.7 – Modèle de fichier `.env.example`

```
SECRET_KEY=votre-cle-secrete-ici  
DEBUG=True  
ALLOWED_HOSTS=localhost,127.0.0.1
```